Basketball Scoreboard: Final Project Report

Tejas Wadhwa
Jean-Milan Albarede
Mehmet Emin Mercan
Hemant Garg
Ian Robert Kozloski

May 2026

Contents

1 Technical

1.1 Project Overview

The goal of this project was to design and build a fully functional basketball scoreboard using digital logic on an FPGA. The system tracks home and away scores, a game clock, a shot clock, ball possession, and the current period. All timing, scoring, and display logic were implemented in SystemVerilog and synthesized onto a Xilinx Arty S7 FPGA development board.

Because the FPGA board lacked built-in seven-segment displays and a buzzer, a custom PCB was designed in KiCad to interface with the FPGA and provide the required output hardware. The PCB included three common-anode seven-segment displays, a piezoelectric buzzer, possession LEDs, and six push buttons for user control.

This project provided experience with the full digital hardware design process, including RTL design, simulation, testbench verification, PCB design, soldering, FPGA integration, and final hardware testing. The completed system operates similarly to a real basketball scoreboard and supports automated timing, scoring, period transitions, and buzzer notifications.

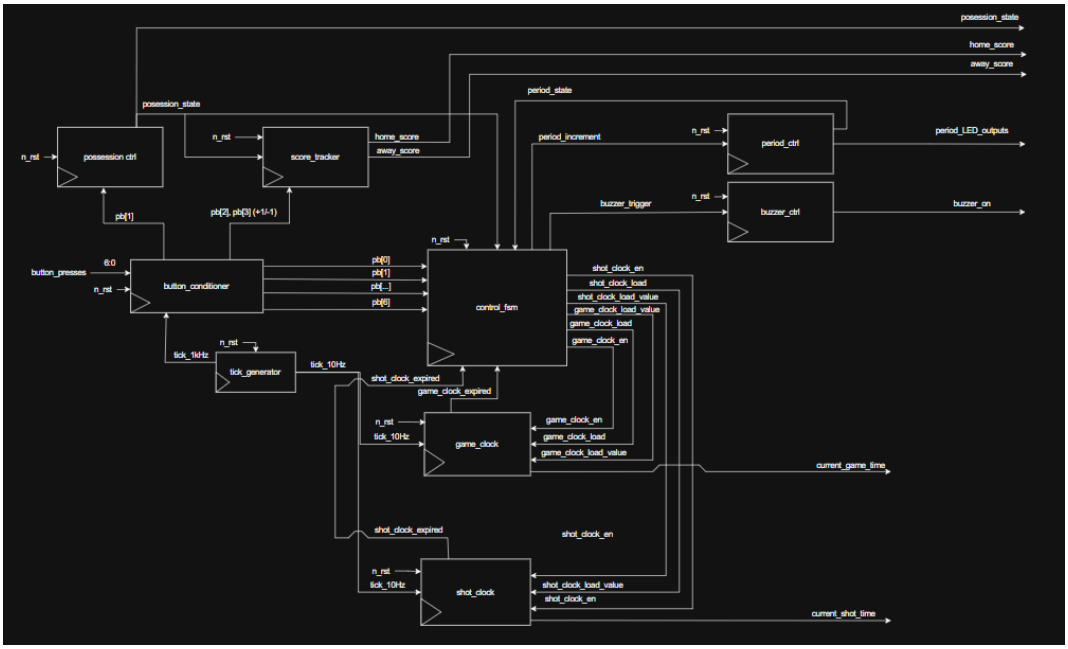


Figure 1: High-Level RTL Diagram of the Basketball Scoreboard System

Figure 1 shows the overall architecture of the basketball scoreboard system. The top-level module coordinates the game clock, shot clock, score tracker, display driver, possession logic, and buzzer control modules. The design is modular so that each component can be individually simulated and tested before system integration.

1.2 Hardware and Software

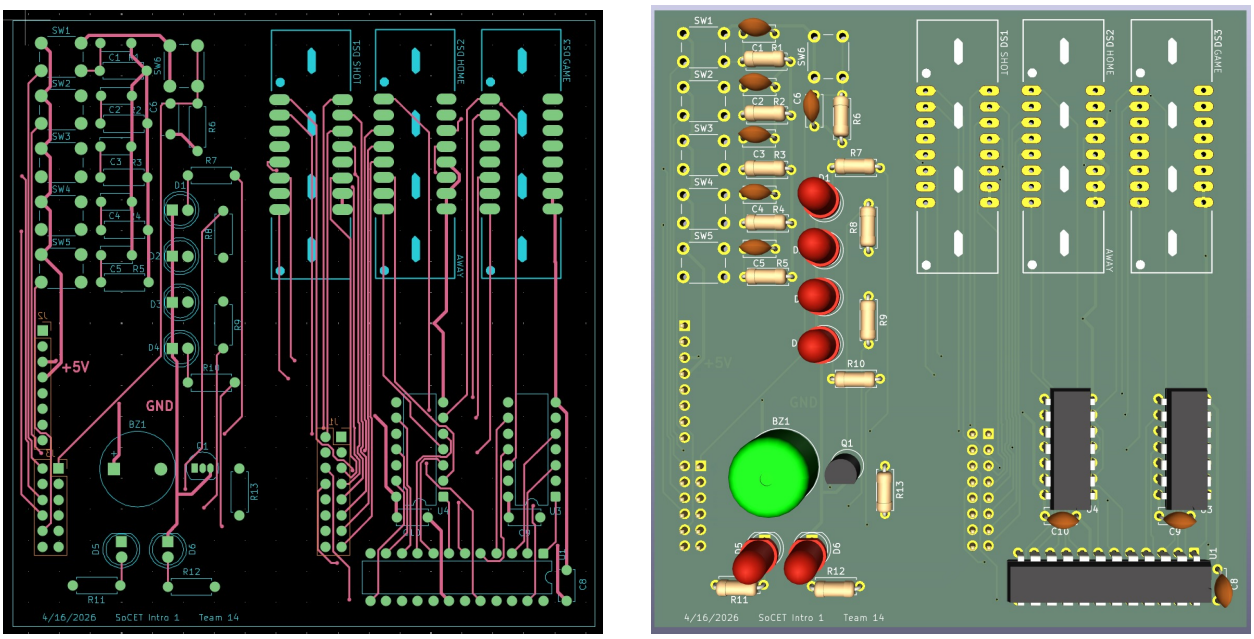
The primary hardware platform used for this project was the Xilinx Arty S7 FPGA development board. A custom PCB was designed to expand the FPGA functionality and provide the external hardware required for the scoreboard system.

Table 1: Hardware Components Used in the Project

Hardware	**Purpose**
Xilinx Arty S7 FPGA	Main digital hardware platform running the SystemVerilog design
Custom PCB	Interfaces displays, LEDs, buzzer, and buttons with the FPGA
Seven-Segment Displays	Displays game clock, shot clock, scores, and period information
Piezoelectric Buzzer	Produces buzzer sound when clocks expire
Push Buttons	User input for scoring, resetting, and possession control
Possession LEDs	Indicates which team currently has possession
4-to-16 Decoder and Inverter ICs	Drives multiplexed seven-segment displays

Table 2: Software and Programming Tools Used

Software	**Purpose**
Vivado	Synthesis, implementation, and FPGA bitstream generation
ModelSim	Behavioral simulation and waveform analysis
KiCad	PCB schematic capture and PCB layout design
Git and GitHub	Version control and collaboration
SystemVerilog	RTL hardware description language used for the project



(a) 2D PCB Layout

(b) 3D PCB View

Figure 2: KiCad Design Perspectives

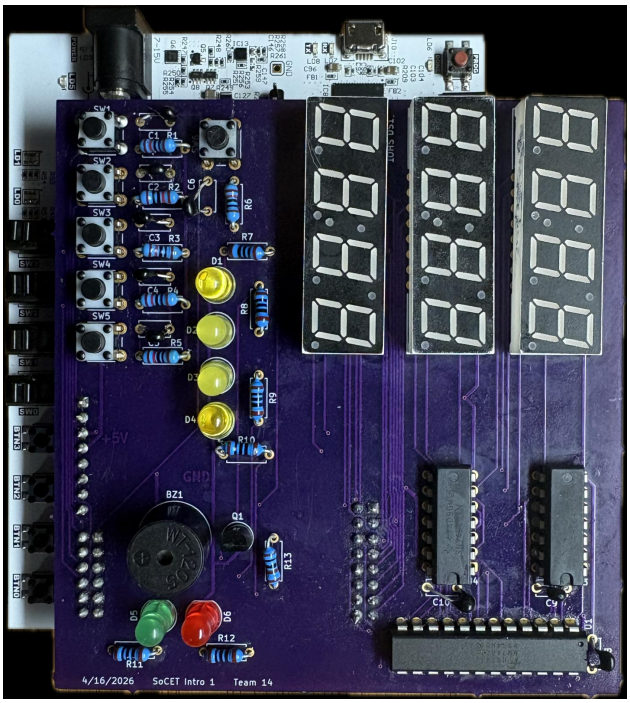


Figure 3: PCB After Soldering and Hardware Assembly

1.3 System Design

The scoreboard system was divided into several major modules, each responsible for a different portion of the design.

1.3.1 Game Clock Module

The game clock module maintains the primary game timer and counts down once per second. The module supports start, stop, pause, and reset operations. When the timer reaches zero, the module signals the control FSM to advance to the next period.

1.3.2 Shot Clock Module

The shot clock module independently tracks the shot timer. The shot clock can be reset manually using the reset button and automatically counts down while the game clock is active.

1.3.3 Score Tracker Module

The score tracker module stores the scores for both teams and updates the correct score depending on the current possession state. The module supports incrementing and decrementing scores while preventing negative score values.

1.3.4 Button Conditioner Module

Mechanical button presses create noisy signals that can cause multiple unintended transitions. The button conditioner module debounces each input and generates clean single-cycle pulses for the rest of the system.

1.3.5 Display Driver Module

The display driver module controls the multiplexing of the seven-segment displays. The module refreshes each display rapidly enough to appear continuously illuminated to the human eye.

1.3.6 Buzzer Driver Module

The buzzer driver module generates an alternating waveform to drive the piezoelectric buzzer. The buzzer activates whenever a timer expires or a period transition occurs.

1.4 Testing and Verification

Each module was verified individually using dedicated SystemVerilog testbenches before full system integration.

- The game clock module was tested for accurate one-second countdown behavior.
- The shot clock module was tested for proper reset and expiration handling.
- The score tracker module was tested for correct score updates and decrement edge cases.
- The display driver module was tested for proper display multiplexing.
- The buzzer driver module was verified to generate the correct waveform and duration.
- The button conditioner module was tested using noisy simulated button presses.

After module-level verification, the complete system was synthesized onto the FPGA and tested in hardware.

1.4.1 Integration Testing

During integration testing, the following behaviors were verified:

- Simultaneous operation of the game clock and shot clock
- Correct score updates for both teams
- Automatic period transitions
- Proper operation of the possession LEDs
- Functional buzzer activation at timer expiration
- Stable seven-segment display operation

1.5 Waveform Results and Benchmarks

The project was verified using waveform simulations using AMD xilinx vivado. The waveforms confirmed correct timer behavior, score updates, and buzzer activation. We had to verify every single module we made and these testbenches are shown in our github. We will not bore you with a bunch of waveforms and we have chosen to just showcase one(they all look the same).

Figure 4: Button conditioner waveform simulation

After detecting a button press for 5 consecutive clock cycles, the conditioned button output goes high for one clock cycle.

The waveform results demonstrated that the system behaved correctly under all tested operating conditions.

1.6 Division of Labor

The project workload was divided based on team member interests and strengths.

Table 3: Division of Labor

Team Member	Contributions
Tejas Wadhwa	Designed the PCB and implemented the top-level FSM and integration logic
Jean-Milan Albarede	Developed the display drivers and button conditioner modules
Mehmet Emin Mercan	Developed the game clock, shot clock, and buzzer driver modules
Hemant Garg	Designed the score tracker module and contributed to PCB layout
Ian Robert Kozloski	Implemented possession logic, period logic, and assisted with PCB soldering

1.7 Project Timeline

The project followed a structured development timeline from planning to final integration.

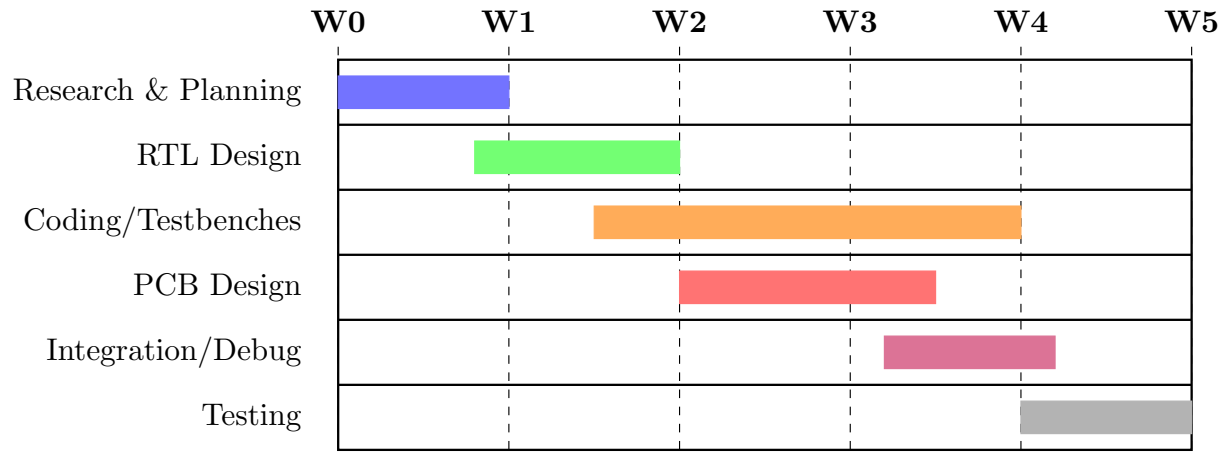


Figure 5: Project Development Timeline

The early stages of the project focused heavily on research, architectural planning, and module decomposition. The middle stages focused on RTL implementation and simulation, while the final stages concentrated on PCB fabrication, integration, debugging, and final testing.

1.8 User Guide

The scoreboard is controlled using six user inputs mapped to FPGA switches and buttons.

Table 4: System Controls

Button	Function
Button 0	Start/Stop game clock and shot clock
Button 1	Toggle possession between teams
Button 2	Add one point to team with possession
Button 3	Subtract one point from team with possession
Button 4	Reset shot clock
Button 5	Full system reset

When the game clock expires, the scoreboard automatically transitions to the next period and activates the buzzer. The possession LEDs indicate which team currently controls the ball.

1.9 Demonstration Video

A full demonstration of the basketball scoreboard system, including the FPGA implementation, PCB hardware, game clock, shot clock, score tracking, possession logic, and buzzer functionality, can be viewed using the link below.

Video Link:
https://purdue0-my.sharepoint.com/personal/jalbared_purdue_edu/_layouts/15/stream.aspx?id=%2Fpersonal%2Fjalbared%5Fpurdue%5Fedu%2FDocuments%2FIntro%20I%20BB%2F20260429%5F213616%2Emp4

1.10 Future Expansion

Several future improvements could significantly expand the project.

- Add configurable game modes for sports other than basketball
- Implement UART or I2C communication for remote control
- Upgrade the display system to LED matrices or LCD displays
- Add foul tracking and bonus indicators
- Add wireless control through Bluetooth or Wi-Fi
- Implement configurable timing rules

2 Reflection

2.1 Learning Outcomes

This project significantly improved our understanding of FPGA-based digital system design, hardware debugging, PCB development, and team collaboration. We learned how to integrate multiple SystemVerilog modules into a complete hardware system and gained practical experience debugging both simulation and physical hardware issues.

The project also strengthened our ability to design testbenches, analyze waveforms, and debug timing-related issues. In addition, the PCB design and soldering process provided valuable hardware experience beyond what is typically covered in standard coursework.

2.2 Team Reflections

2.2.1 Tejas Wadhwa

Working on the control FSM and top-level module taught me how to coordinate multiple hardware subsystems, something our class assignments never required at this scale. I had to think about how signals flow between modules and how timing dependencies create unexpected behavior when everything runs together. I also gained my first real experience with PCB design in KiCad. What I enjoyed most was the early design phase, where we sketched RTL diagrams as a team before anyone wrote code. The most frustrating part was debugging integration issues during the final week, especially the 5V-to-GPIO schematic mistake that fried several pins and forced us to remap our assignments under a tight deadline.

2.2.2 Jean-Milan Albarede

My biggest takeaway was understanding the interface between digital logic and physical hardware. Writing the display driver modules meant I had to think about how the 4-to-16 decoder, inverter IC, and common-anode displays interact, and how refresh rates affect brightness at a 1/11 duty cycle. The button conditioner module taught me about debouncing on real, noisy signals. Seeing the displays light up for the first time after weeks of simulation waveforms was genuinely satisfying. What I disliked was dealing with hardware surprises that simulation couldn't predict, like the linked colon segments on the displays and the tedious trial-and-error of tuning refresh rates. The fried GPIO pins reinforced how a single schematic oversight can cascade into a serious hardware failure.

2.2.3 Mehmet Emin Mercan

This project was my first time writing SystemVerilog for hardware that would actually run. Building the game clock, shot clock, and buzzer driver modules taught me how critical it is to consider edge cases and timing from the start. Writing testbenches initially felt like a chore, but they saved enormous time during integration, since I could reproduce bugs in isolation and fix them with confidence. I enjoyed the puzzle of the clock modules, figuring out internal time storage, display formatting, and accurate countdown using the tick generator. What I disliked was Vivado's long synthesis times, which slowed debugging considerably. My biggest challenge was the buzzer driver: the buzzer only activates with AC, so the output waveform and duration parameter both had to be precise.

2.2.4 Hemant Garg

The score tracker module deepened my understanding of state management in hardware. Routing score updates to the correct register based on the possession signal, and handling both increment and decrement cleanly, required more careful thinking than I expected. I also contributed to the PCB layout in KiCad, which was my first time placing components and routing traces for a real fabrication run. I enjoyed the collaborative structure of our team; weekly meetings and well-defined module interfaces let me work independently without worrying about conflicts. What I disliked was the stress of discovering fried GPIO pins during integration and scrambling to remap everything. My biggest challenge was handling edge cases like preventing the score from going negative on a decrement at zero.

2.2.5 Ian Robert Kozloski

I wrote the possession control, period control, and buzzer control modules, and I also soldered and assembled the PCB. The modules were relatively small, but they taught me how even simple state machines need careful design when they interact with a larger system. Soldering the PCB was my favorite part of the project. Taking a bare board and populating it with components until it functions is deeply satisfying. What I disliked was troubleshooting cold solder joints; one display segment wouldn't light up, and tracking that to an incomplete joint that looked fine visually took longer than I'd like to admit. The main challenge was helping remap pin assignments after the voltage issue fried several GPIOs, which gave me a real appreciation for double-checking every part of a schematic before fabrication.

3 Conclusion

The basketball scoreboard project successfully demonstrated the complete FPGA hardware design flow, including RTL implementation, simulation, PCB design, hardware integration, and debugging. The final system met the intended functionality requirements and provided valuable experience in both digital logic and hardware development.

The project combined concepts from SystemVerilog, FPGA design, simulation, hardware interfacing, and PCB fabrication into a single integrated design experience. Overall, the project provided a strong foundation for future work in digital systems, FPGA development, and embedded hardware design.

GitHub Repository

The complete project repository, including all SystemVerilog source files, PCB design files, simulations, and documentation, is available below:

https://github.com/wadhwat/socet-1-shot-clock